Pre-Reads: OOPs Concepts - Basics

Nishut Suman 17 Dec, 2025 At 8:00 AM Pre-Reads Foundation Recommended Resource

Description Discussions

Pre-Read: OOPs Basics

🧠 What You'll Gain from This Pre-Read

After reading this, you'll be able to:

- Understand **why** Object-Oriented Programming (OOP) is used.
- Recognize what **classes** and **objects** are in Python.
- Know what **constructors**, **attributes**, and **methods** mean.
- Compare **procedural** vs **object-oriented** programming.
- Create simple Python classes and objects yourself.

Think of this as:

> Learning how to organize your ideas and code like a real-world architect instead of just building walls randomly.

🌍 Part 1: The Big Picture - Why Does OOP Matter?

Imagine you're building a game 🎮 . You need players, enemies, weapons, and scores. If you write separate functions for each player, you'll repeat lots of code-messy and hard to manage.

That's where **Object-Oriented Programming** comes to the rescue 💡 . OOP helps you bundle **data** (attributes) and **actions** (methods) together into a simple package called a **class**.

Where You'll Use This:

- **Backend developers**: Designing user, product, or order models in apps.
- **Game developers**: Defining characters, weapons, and behaviors.
- **AI/ML developers**: Organizing models and data pipelines.

Real Products:

- Instagram's backend uses OOP to handle posts, users, and messages.

- Netflix models shows, users, and recommendations as objects.
- Spotify treats songs and playlists as objects that interact.

Think of it like this:

A **class** is a **blueprint** (like a car design 🚗).

An **object** is the **real car** built from that blueprint (like *my_tesla*).

Limitation: A blueprint can describe many cars, but each car has its own color, speed, and mileage - so every object has unique data.

🗺️ Part 2: Your Roadmap Through This Topic

We'll explore:

Understanding why OOP is needed
Exploring classes and objects
Using constructors (***init**) (Here use 2 underscores only each side)
Working with attributes and methods
Comparing procedural **vs** Object-oriented style

By the end, you'll not just *know* OOP - you'll be able to *think* in **OOP** terms.

🏗️ Part 3: Key Terms to Listen For

Class

A **blueprint** for creating objects.

Example: `class Car:` defines what a car should have (like color, model, and speed).

Object

An **instance** of a class - a real thing created from the blueprint. **Example:** `my_car = Car()` makes one actual car from the class Car.

Constructor (*init*)

A **special method** that automatically runs when you create an object. It sets up the object's data.

Example: Giving your car its color and model when you build it.

Attributes

Data or properties that belong to an object.

Example: `self.color, self.model`

Methods

Functions that describe an object's behavior.

Example: `drive() or brake()`

💡 **Key Insight:** Classes combine data and behavior - like **nouns and verbs** together in one sentence.

💻 Part 4: Concepts in Action

🧩 Understanding Classes and Objects

Scenario: You want to model a dog for a pet app 🐶 .

Our approach: We'll define a class **Dog** with attributes (name, breed) and a method (**bark**).

Code Example:

```
Step 1: Define a blueprint for a Dog
class Dog:

    Step 2: Constructor - defines what happens when we create a Dog
    def __init__(self, name, breed):
        self.name = name      # attribute 1
        self.breed = breed    # attribute 2

    Step 3: Define a behavior
    def bark(self):
        print(f"{self.name} says Woof!")

Step 4: Create (instantiate) objects from the class
dog1 = Dog("Buddy", "Golden Retriever")
dog2 = Dog("Luna", "Beagle")

Step 5: Call the method
dog1.bark()
dog2.bark()
```

Output:

```
Buddy says Woof!
Luna says Woof!
```

What's happening here:

- **class Dog:** → defines a new type of object.
- ***init*()** → sets up data for each object.
- **self** → refers to the current object.
- **dog1** and **dog2** → two unique dogs, created from the same class.

Key takeaway: You don't just write code - you design **thing** that act like real-world objects.

⚖️ Part 5: Procedural vs Object-Oriented Programming

Aspect	Procedural	Object-Oriented
Style	Step-by-step instructions	Models of real-world entities
Data Handling	Functions and data are separate	Data and functions live together
Example	calculate_area(length, width)	rectangle.area()
Reusability	Low	High
Scalability	Hard to manage	Easy to extend

Mini Example:

```
# Procedural
def student_intro(name, gpa):
    print(f"Hi, I'm {name} and my GPA is {gpa}")

student_intro("Asha", 9.0)

# Object-Oriented
class Student:
    def __init__(self, name, gpa):
        self.name = name
        self.gpa = gpa

    def introduce(self):
        print(f"Hi, I'm {self.name} and my GPA is {self.gpa}")

s1 = Student("Asha", 9.0)
s1.introduce()
```

Output:

```
Hi, I'm Asha and my GPA is 9.0
```

✅ The OOP version is cleaner, easier to expand, and feels like describing *real students* instead of just running code.

🪄 Part 7: Common Beginner Mistakes

Mistake	Wrong Code	Correct Code
Forgetting **self**	def bark():	def bark(self):
Not using constructor properly	Dog("Max") without matching ***init*** args	Dog("Max", "Bulldog")
Mixing attributes	print(name)	print(self.name)
Using capitalized method names (Java habit)	def Bark(self):	def bark(self):

🎯 Part 8: How This Connects

Builds on:

- Variables, functions, and logical flow.

Enables:

- Building scalable applications.
- Reusing code easily.
- Implementing inheritance, polymorphism, and encapsulation later.

Next Step: → Dive into **Encapsulation and Inheritance** (the next OOP pillars).

💬 Part 9: Reflect & Explore

Think:

- How is a class like a blueprint in your daily life (school ID, bank account, or car)?
- What's one example of an object you interact with daily?

Try: Create a class **Book** with **title**, **author**, and **price**, and a method **info()** that prints these details.

🌟 Final Thought

> Procedural code tells the computer what to do. Object-Oriented code teaches the computer how to think about the world.